

Analiza SLURM oraz architektury systemu

Projekt: Koło Naukowe Data Science

1. Proponowana architektura systemu

1.1 Ogólne założenia

Zbieraj celowo Monitoring jest powiązany z cyklem życia Slurm - uruchamia się przy starcie joba, kończy się przy jego zakończeniu. Nie chcemy zbierać danych o procesach, których nie analizujemy.

Ufajmy Slurm. Dane o jobach (czasy, zadeklarowane zasoby, stan końcowy) pochodzą wyłącznie z `slurmdb` - systemowego komponentu Slurma. Nie duplikujemy tej funkcji własnym kode.

Dostarcz informacje Naukowiec powinien otrzymać raport efektywności automatycznie po zakończeniu zadania, a przed jego uruchomieniem konkretna sugestię parametrów `#SBATCH` opartą na własnej historii.

1.2 Warstwy systemu

System składa się z czterech warstw o rozdzielonych odpowiedzialnościach:

Warstwa zbierania danych działa na każdym węźle GPU. Zamiast `nvidia-smi` wywoływanego przez `cron`, używamy NVIDIA Data Center GPU Manager (DCGM) demona systemowego utrzymującego stałe połączenie ze sterownikami NVIDIA. DCGM dostarcza metryki z sub-sekundową rozdzielczością przy narzucie poniżej 0,1% CPU, a co najważniejsze automatycznie oznacza metryki identyfikatorem zadania Slurm po rejestracji przez skrypt Prolog. Obok DCGM każdy węzeł uruchamia `node_exporter` do metryk systemowych (CPU, RAM, sieć).

Warstwa przechowywania składa się z dwóch komponentów o różnych rolach. Prometheus zbiera metryki z węzłów co 15 sekund i przechowuje je przez 90 dni pełni rolę bufora danych w czasie rzeczywistym, zasilającego alerty i dashboardy Grafana dla administratorów. PostgreSQL z rozszerzeniem TimescaleDB przechowuje dane o zadaniach i telemetrię w ustrukturyzowanej formie dostępnej dla API i modułu estymacji zasobów. W późniejszych sekcjach znajdzie się uzasadnienie wyboru.

Warstwa analizy to zestaw serwisów działających w tle na serwerze zarządzającym. Po zakończeniu zadania `efficiency_scorer` pobiera metryki z Prometheus, łączy je z danymi z `slurmdb` i oblicza wynik efektywności w skali 0–100. Osobny serwis `idle_gpu_monitor` sprawdza co kilka minut czy aktywne zadania efektywnie używają GPU, jeśli nie, wysyła ostrzeżenie emailiem do użytkownika.

Warstwa prezentacji ma dwa osobne interfejsy. Grafana dostępna dla administratorów wydziału pokazuje stan klastra, statystyki użycia i historyczne wykresy. Dodatkowym pomysłem jest stworzenie własnej aplikacji webowej skierowana do naukowców udostępnia trzy funkcje: podgląd stanu klastra (które GPU są zajęte i przez kogo), historię własnych zadań z ocenami efektywności oraz estymator zasobów formularz generujący konkretny fragment `#SBATCH` na podstawie historii podobnych zadań. Natomiast jest to pomysł raczej na później.

--

2. Integracja ze Slurm

Slurm to system zarządzania zasobami z rozbudowanym modelem rachunkowości, punktami wejścia dla zewnętrznych skryptów i mechanizmami śledzenia zasobów sprzętowych. Poniżej opisujemy każdy z mechanizmów, który uwzględniliśmy w projekcie, wraz z uzasadnieniem dlaczego go wybraliśmy lub odrzuciliśmy.

2.1 Architektura Slurma - komponenty istotne dla projektu

Slurm składa się z kilku demonów (daemons) o ściśle rozdzielonych odpowiedzialnościach.

slurmctld (kontroler) działa na jednym serwerze zarządzającym i jest centralnym mózgiem systemu. Przyjmuje zgłoszenia zadań, zarządza kolejką i przydziela zasoby węzłom. Konfiguruje się w nim globalne polityki kolejkowania i priorytety.

slurmd (demon węzłowy) działa na każdym węźle obliczeniowym. Otrzymuje polecenia od **slurmctld**, uruchamia zadania, monitoruje ich stan i zgłasza zakończenie. To **slurmd** wywołuje skrypty Prolog i Epilog bezpośrednio na węźle, na którym uruchomione jest zadanie.

slurmdbd (daemon bazy danych) to osobny proces odpowiedzialny wyłącznie za rachunkowość — przechowuje historię wszystkich zadań, użytkowników, alokacji i zasobów w relacyjnej bazie danych (domyślnie MariaDB/MySQL). Jest to jedyne autorytatywne źródło danych o tym co, kiedy i przez kogo było uruchomione na klastrze.

sacct to narzędzie wiersza poleceń do odpytywania **slurmdbd**. Pozwala pobierać szczegółowe dane o zakończonych i bieżących zadaniach. Używamy go w serwisie **job_collector** do cyklicznej synchronizacji danych z naszą bazą analityczną.

2.2 Śledzenie GPU przez Slurm

Slurm zarządza GPU jako tzw. Generic Resources (GRES). Każda karta GPU musi być zadeklarowana w pliku **gres.conf** na węźle. Slurm może wtedy ją przydzielać, ograniczać widoczność i raportować zużycie.

Gdy użytkownik deklaruje **-gres=gpu:2**, Slurm wykonuje kilka czynności jednocześnie: rezerwuje dwie konkretne karty GPU na czas trwania zadania, ustawia w środowisku procesu zmienną **CUDA_VISIBLE_DEVICES** z indeksami przydzielonych kart (np. **CUDA_VISIBLE_DEVICES=2,5**) oraz zapisuje informacje o przydziale w **slurmdbd**. Zmienna **CUDA_VISIBLE_DEVICES** jest mechanizmem, przez który CUDA decyduje, które karty są widoczne dla programu — jeśli jest ustawiona przez Slurm, program nie może *przypadkowo* użyć karty innego użytkownika.

Istotne jest, że Slurm zna dokładnie które karty GPU przydzielił danemu zadaniu — ta informacja jest dostępna w zmiennej środowiskowej **SLURM_JOB_GPUS** wewnątrz skryptów Prolog i Epilog. Właśnie to wykorzystujemy do rejestracji w DCGM.

Aby **slurmdbd** przechowywał faktyczne zużycie zasobów GRES (nie tylko zadeklarowane), w konfiguracji Slurma musi być ustawione **AccountingStorageTRES=gres/gpu**. Po zakończeniu zadania **sacct** udostępnia wtedy pole **GresUsed**, które zawiera rzeczywiste użycie GPU zgłoszone przez węzeł — niezależnie od tego co mierzy DCGM. Mamy zatem dwa niezależne źródła informacji o zużyciu GPU, co pozwala je wzajemnie weryfikować.

2.3 Mechanizm Prolog i Epilog

Prolog i Epilog to skrypty powłoki wywoływane przez `slurmd` przy każdym starcie i zakończeniu zadania. Są rejestrowane w `/etc/slurm/slurm.conf` dyrektywami `Prolog=` i `Epilog=`. To najprostszy i najbezpieczniejszy punkt wejścia dla zewnętrznej integracji — nie wymagają modyfikacji kodu Slurma ani kompilacji wtyczek.

Kontekst wykonania. Domyślnie Prolog i Epilog uruchamiane są jako użytkownik `root` na każdym węźle, na którym działa zadanie. To daje pełne uprawnienia do modyfikacji środowiska węzła — w naszym przypadku potrzebujemy ich do zapisu pliku mapowania w katalogu `/run/dcgm-exporter/job-mapping/`. Alternatywnie można użyć dyrektywy `PrologFlags=Allnodes`, która uruchamia Prolog na wszystkich węzłach równocześnie, lub `PrologFlags=Serial` wymuszającej sekwencyjne wywołania.

Dostępne zmienne środowiskowe. W momencie wywołania Prologu dostępne są kluczowe zmienne ustawiane automatycznie przez Slurm:

- `SLURM_JOB_ID` — unikalny identyfikator zadania (klucz integracji z resztą systemu)
- `SLURM_JOB_USER` — nazwa użytkownika, który zgłosił zadanie
- `SLURM_JOB_GPUS` — lista indeksów GPU przydzielonych zadaniu na tym węźle (np. `2,5`)
- `SLURM_JOB_NODELIST` — węzły przydzielone zadaniu
- `SLURM_JOB_PARTITION` — partycja, na której uruchomiono zadanie
- `SLURM_JOB_NUM_NODES` — liczba węzłów
- `SLURM_SUBMIT_DIR` — katalog roboczy z momentu zgłoszenia
- `SLURM_JOB_SUBMIT_TIME`, `SLURM_JOB_START_TIME` — czasy w formacie epoch

W Epilogu dodatkowo dostępne są `SLURM_JOB_END_TIME` oraz `SLURM_JOB_EXIT_CODE` — kod wyjścia procesu użytkownika.

Wymagania dotyczące kodu wyjścia. Prolog *musi* zwrócić kod 0. Jeśli zwróci cokolwiek innego, `slurmctld` natychmiast anuluje zadanie i oznacza węzeł jako niedostępny (`drain`). To ograniczenie projektowe, ponieważ każda operacja w Prologu, która może zawieść, musi być opakowana w obsługę błędów kończącą się sukcesem. W naszej implementacji awaria zapisu do bazy danych jest logowana, ale nie powoduje błędu Prologu. Zadanie użytkownika musi się uruchomić niezależnie od stanu naszego systemu monitorowania.

Epilog jest asynchroniczny względem zadania, ale synchroniczny względem zwalniania zasobów. `slurmd` czeka na zakończenie Epilogu zanim zgłosi węzeł jako wolny do `slurmctld`. Długi Epilog opóźnia więc przydzielenie węzła kolejnemu zadaniu. Z tego powodu obliczenie wyników efektywności, które wymaga zapytania do Prometheus i sacct uruchamiamy w tle (`&`), a sam Epilog kończy się natychmiast po zarejestrowaniu zdarzenia.

Prolog slurmctld vs Prolog węzłowy. Istnieje też dyrektywa `PrologSlurmctld=` — skrypt wywoływany po stronie kontrolera, a nie węzła, z prawami `SlurmUser`. Uruchamia się tylko raz dla całego zadania, nawet jeśli obejmuje wiele węzłów. Rozważaliśmy go jako alternatywę dla centralnego rejestrowania startu zadania, ale zdecydowaliśmy się na węzłowy Prolog, ponieważ rejestracja mapowania DCGM musi nastąpić na węźle (gdzie działa dcgm-exporter), nie na kontrolerze.

2.4 Korelacja GPU <-> zadanie - problem i rozwiązanie

Narzędzia takie jak `nvidia-smi` operują na poziomie procesów systemu operacyjnego (PID) — nie wiedzą nic o Slurm. Aby powiązać metryki GPU z konkretnym zadaniem, potrzebujemy mostu między światem procesów a światem identyfikatorów Slurm.

Podejście poprzednie (Dynamic-Monitoring): czytanie `/proc/<PID>/environ` dla każdego procesu, szukanie `SLURM_JOB_ID` i `CUDA_VISIBLE_DEVICES`. To podejście ma kilka wad: wymaga uprawnień do odczytu środowisk cudzych procesów, działa z opóźnieniem (cron co minutę), i jest podatne na race conditions gdy zadanie kończy się w trakcie skanowania.

Podejście DCGM Job Mapping: DCGM exporter obsługuje mechanizm mapowania zadań przez katalog `/run/dcgm-exporter/job-mapping/`. Gdy Prolog zapisuje do tego katalogu plik o nazwie odpowiadającej identyfikatorowi zadania Slurm, zawierający listę przydzielonych indeksów GPU, dcgm-exporter automatycznie dodaje etykietę `slurm_job_id` do wszystkich metryk zbieranych z tych kart. Mapowanie jest aktywne od razu, bez żadnego opóźnienia, i nie wymaga skanowania procesów.

Przepływ jest następujący: Prolog zapisuje plik `/run/dcgm-exporter/job-mapping/735540` zawierający `2,5`. Od tej chwili wszystkie metryki DCGM zbierane z GPU o indeksach 2 i 5 na tym węźle mają w Prometheus etykietę `slurm_job_id="735540"`. Epilog usuwa ten plik, co natychmiast kończy oznaczanie. Rozwiązuje to problem korelacji dokładnie, bez opóźnienia i bez uprawnień do skanowania procesów.

2.5 slurmdbd i sacct jako źródło danych accountingowych

`slurmdbd` przechowuje dane o każdym zadaniu w relacyjnej bazie danych. Dostęp do tych danych odbywa się przez polecenie `sacct`. Dla naszego systemu kluczowe są następujące pola:

- `JobID` — identyfikator zadania (klucz integracji)
- `User` — użytkownik
- `Partition` — partycja kolejki
- `Submit`, `Start`, `End` — czasy w formacie ISO 8601
- `Elapsed` — rzeczywisty czas trwania
- `State` — stan końcowy (`COMPLETED`, `FAILED`, `CANCELLED`, `TIMEOUT`, `OUT_OF_MEMORY`)
- `ReqGRES` — zadeklarowane zasoby GPU (np. `gpu:a100:2`)
- `GresUsed` — faktycznie użyte zasoby GPU (dostępne gdy `AccountingStorageTRES=gres/gpu`)
- `ReqCPUS`, `ReqMem` — zadeklarowane CPU i pamięć
- `MaxRSS` — szczytowe użycie pamięci RAM (z monitorowania `slurmstepd`)
- `TotalCPU` — sumaryczny czas CPU zużyty przez zadanie
- `WorkDir` — katalog roboczy

Porównanie `ReqGRES` z `GresUsed` pozwala wykryć zadania, które zadeklarowały więcej GPU niż faktycznie użyły. Jest to niezależne potwierdzenie tego, co widzimy w metrykach DCGM. Jeśli `sacct` raportuje `GresUsed=gpu:0` przy `ReqGRES=gpu:4`, zadanie zadeklarowało GPU, ale ich w ogóle nie użyło, to ostrzejszy przypadek niż niska użycia.

Nasz serwis `job_collector` odpytuje `sacct` co pięć minut z parametrem `-starttime` wskazującym na ostatnią godzinę. Używa trybu `-parsable2` (pola oddzielone `|`) dla niezawodnego parsowania. Nowe i zaktualizowane rekordy są upsertowane do tabeli `job_runs` w PostgreSQL, jeśli rekord istnieje (wstawiony przez Prolog), aktualizuje tylko pola które zna `slurmdbd`, nie nadpisując metryk z DCGM.

2.6 Mechanizmy ocenione i odrzucone

W trakcie badań przeanalizowaliśmy kilka dodatkowych mechanizmów integracji ze Slurm, które ostatecznie odrzuciliśmy. Opisujemy je tutaj, ponieważ ich odrzucenie jest równie istotną decyzją projektową co wybór mechanizmów wdrożonych.

AcctGatherProfile to wtyczka Slurma (`acct_gather_profile/influxdb` lub `acct_gather_profile/hdf5`) która sprawia, że `slurmstepd` — demon zarządzający krokami zadania — samodzielnie próbuje zużycie CPU, RAM i I/O i wysyła dane bezpośrednio do zewnętrznej bazy. Mechanizm jest atrakcyjny, ponieważ zbiera dane z wewnątrz Slurma, bez dodatkowych agentów. Ma jednak istotne ograniczenia: zbiera wyłącznie metryki systemowe (CPU, RAM, sieć, I/O dysku), nie metryki GPU — DCGM i tak jest potrzebny osobno. Wersja HDF5 produkuje pliki na współdzielonym systemie plików przy każdym kroku zadania, co przy macierzy DDN obsługującej wiele zadań jednocześnie generuje obciążenie metadana serwera. Wersja InfluxDB wymaga wtyczki skompilowanej razem ze Slurm w konkretnej wersji. Biorąc pod uwagę, że DCGM + Prometheus pokrywa nasze potrzeby z lepszą rozdzielczością i bez ryzyka obciążenia systemu plików, zdecydowaliśmy się nie wdrażać AcctGatherProfile.

`job_submit.lua` to wtyczka po stronie `slurmctld` wywoływana w momencie przyjęcia zgłoszenia zadania przez użytkownika. Pozwala walidować i modyfikować parametry `#SBATCH` przed wstawieniem zadania do kolejki, na przykład odrzucać zadania bez limitu czasu albo automatycznie kierować małe zadania na partycję Pascal. Ryzykiem jest jednak to, że błąd składniowy lub wyjątek w skrypcie LUA powoduje, że `slurmctld` odrzuca wszystkie nowe zgłoszenia zadań do momentu naprawy — efektywnie blokując klaster. Bez dedykowanego środowiska testowego z identyczną wersją Slurm i bez możliwości rollbacku w kilkanaście sekund, ryzyko to jest nieakceptowalne. Zostawiamy `job_submit.lua` jako potencjalne rozszerzenie na przyszłość.

Wtyczki SPANK (Slurm Plug-in Architecture for Node and job Kontrol) to biblioteki dynamiczne w C ładowane przez `slurmstepd` i `srun`. Pozwalają dodawać nowe opcje wiersza poleceń Slurma, modyfikować środowisko uruchomieniowe zadania i reagować na zdarzenia lifecycle. Są najbardziej elastycznym mechanizmem rozszerzania Slurma. Wymagają jednak kompilacji C, linkowania z bibliotekami Slurma w konkretnej wersji i instalacji na każdym węźle. Błąd w wtyczce SPANK może crashować `slurmstepd`, co oznacza utratę wszystkich aktywnych zadań na węźle. Dla naszych celów rejestracji zdarzeń i mapowania GPU — Prolog/Epilog w Bashu jest wystarczający i bezpieczniejszy.

2.7 Zmiany konfiguracji Slurma wymagane przez projekt

W pliku `/etc/slurm/slurm.conf` należy dodać lub uaktualnić:

```
# Rejestracja skryptów lifecycle
Prolog=/opt/eden-monitor/slurm-hooks/prolog.sh
Epilog=/opt/eden-monitor/slurm-hooks/epilog.sh

# Rachunkowość GPU – bez tego sacct nie raportuje GresUsed
AccountingStorageTRES=gres/gpu,cpu,mem

# Typ zasobów generycznych – musi być zgodne z gres.conf
GresTypes=gpu
```

```
# Monitorowanie zużycia CPU i pamięci przez slurmstepd
JobAcctGatherType=jobacct_gather/linux
JobAcctGatherFrequency=30
```

Dyrektywa `AccountingStorageTRES=gres/gpu` jest szczególnie istotna — bez niej `slurmdbd` nie przechowuje informacji o faktycznym zużyciu GPU i pole `GresUsed` w `sacct` pozostaje puste. To sprawia, że porównanie *zadeklarowane vs użyte* na poziomie Slurma jest niemożliwe.

Po dodaniu `Prolog=` i `Epilog=` wymagany jest restart `slurmctld`. Restart nie przerywa działających zadań — `slurmd` na węzłach kontynuuje ich wykonanie. Nowe Prolog/Epilog zaczną być wywoływane przy kolejnych zadaniach startujących po restarcie.

3. Wybór bazy danych

Poniżej przedstawiamy wnioski i ich uzasadnienie w kontekście całej architektury.

3.1 Rozważane opcje

Analizowaliśmy trzy kandydatury: PostgreSQL z rozszerzeniem TimescaleDB, ClickHouse oraz InfluxDB.

ClickHouse to baza kolumnowa zoptymalizowana pod szybkie zapytania analityczne na bardzo dużych wolumenach danych. Jest jednak zoptymalizowana pod model *append-only* - operacja `UPDATE` (którą wykonujemy po każdym zakończonym zadaniu, uzupełniając `efficiency_score`) jest w niej skomplikowana i kosztowna. Biorąc pod uwagę, że nasz model danych wymaga zarówno dopisywania nowych rekordów jak i aktualizowania istniejących, ClickHouse odpada jako kandydat.

InfluxDB jest bazą dedykowaną do metryk czasowych i dobrze sprawdza się w czystych scenariuszach monitoringowych. Jej wadą jest słaba obsługa zapytań relacyjnych — zapytanie przekrojowe takie jak „*które zadania użytkownika X na partycji Y w ciągu ostatnich 30 dni miały efektywność poniżej 40%*” wymaga łączenia danych z kilku miejsc. W praktyce oznaczałoby to konieczność utrzymania drugiej bazy na metadane, co zwiększa złożoność systemu bez proporcjonalnych korzyści.

PostgreSQL + TimescaleDB to jedyne rozwiązanie pokrywające oba przypadki użycia jednocześnie: transakcyjne operacje na metadanych zadań oraz efektywne przechowywanie szeregów czasowych. TimescaleDB jako rozszerzenie do PostgreSQL nie wymaga osobnego serwera - instaluje się jako pakiet i aktywuje jednym poleceniem SQL. Dla tabeli metryk (`node_snapshots`) tworzymy tzw. hypertable, którą TimescaleDB automatycznie dzieli na chunki po czasie bez żadnego dodatkowego kodu. Retencja (automatyczne usuwanie danych starszych niż 6 miesięcy) i downsampling (zastępowanie próbek co 5 minut agregatami godzinowymi po 30 dniach) konfiguruje się jednym poleceniem SQL każde.

3.2 Podział odpowiedzialności: slurmdb i TimescaleDB

`slurmdbd` jest autorytatywnym źródłem wiedzy o zadaniach Slurma. Dostarcza identyfikatory zadań, użytkowników, partycje, stany końcowe oraz czasy i zasoby zadeklarowane przez użytkownika. Te dane są niezbędne do rzetelnego porównywania *zadeklarowane vs użyte* i do budowy rekomendacji zasobów. Ufamy Slurm accounting — nie reimplementujemy tej funkcji własnym kodem.

TimescaleDB przechowuje to, czego `slurmdbd` nie ma w wymaganej szczegółowości: metryki GPU/CPU/RAM zbierane cyklicznie, zdarzenia nieefektywności z rozdzielczością na sekundy, obliczone

wyniki efektywności i historię sugestii zasobów.

Integracja obu warstw odbywa się przez klucz `slurm_job_id`. Praktyczną zaletą tego podziału jest odporność na awarie naszego systemu: jeśli przez kilka godzin nie działamy, `slurmdbd` zachowuje kompletną historię zadań, którą możemy zaimportować przez `sacct` bez utraty danych.

3.3 Zasady zarządzania danymi

Retencja - surowe snapshoty GPU starsze niż 6 miesięcy są automatycznie usuwane przez politykę retencji TimescaleDB. Konfiguracja to jeden wiersz SQL, wykonanie jest automatyczne.

Downsampling - po 30 dniach surowe próbki co 5 minut są zastępowane przez agregaty godzinowe (średnia, maksimum, procent bezczynności). TimescaleDB utrzymuje i odświeża te agregaty automatycznie przez mechanizm Continuous Aggregates.

Logowanie zdarzeń, nie próbek - zdarzenia nieefektywności (takie jak „GPU bezczynne przy aktywnym zadaniu”) zapisujemy wyłącznie jako pary begin/end, nie jako próbkę przy każdym odczycie. To kilka rekordów zamiast tysięcy.

Partycjonowanie po czasie - TimescaleDB automatycznie dzieli hypertable na chunki tygodniowe. Operacje retencji polegają wtedy na usunięciu całego chunka, nie na skanie i usunięciu wierszy, są wielokrotnie szybsze.

--

4. Estymator zasobów

Estymator zasobów to nowa funkcja, której obecny system nie oferuje w żadnej formie. Jest jednocześnie najbardziej wartościową z punktu widzenia celu projektu.

Użytkownik wypełnia formularz w interfejsie webowym: wybiera framework (PyTorch, TensorFlow, własny kod CUDA), typ zadania (trening, inferencja, symulacja), szacowany rozmiar danych i opcjonalnie liczbę parametrów modelu. System przeszukuje bazę historycznych zadań tego użytkownika o podobnym profilu i oblicza medianą zużytych zasobów. Mediana jest bardziej odporna na wartości odstające niż średnia, co jest istotne — pojedyncze bardzo długie lub bardzo krótkie zadanie nie powinno skrzywić sugestii. Do czasu dodajemy margines bezpieczeństwa 20%, ponieważ błąd w kierunku za mało czasu jest bardziej kosztowny niż za dużo.

Wynikiem jest gotowy fragment `#SBATCH` do skopiowania, wraz z informacją o liczbie historycznych zadań użytych jako podstawa sugestii i przykładami kilku z nich. Jeśli użytkownik nie ma historii (pierwsze uruchomienie na klastrze), system korzysta z danych globalnych — podobnych zadań innych użytkowników.

Sugestie są z definicji przybliżone. Ich wartość nie polega na precyzji, lecz na przekształceniu domysłu „może dwa GPU i dobę” w szacunek oparty na danych „twoje poprzednie podobne zadania zajmowały medianę 1,8 GPU przez 5,4 godziny”. Po uruchomieniu zadania rzeczywiste zużycie jest zapisywane, co poprawia jakość przyszłych sugestii.

Dodatkowo można dodać wcześniej rozważany pomysł, ze skryptem testującym algorytm na procencie danych i estymacji potrzebnych zasobów na tej podstawie.

--

5. Czego nie robimy i dlaczego

Kilka elementów analitycznych zostało świadomie wyłączonych z zakresu projektu.

Izolacja zasobów przez cgroups v2 i eBPF to mechanizm zapobiegający dostępowi zadań do GPU, których Slurm im nie przydzielił. Jego konfiguracja wymaga głębokiej wiedzy o jądrze Linux i błąd może spowodować niestabilność węzła. Nasze rozwiązanie nie wymaga tej izolacji do prawidłowego działania.

Partycjonowanie MIG na kartach A100 pozwala podzielić jedną fizyczną kartę A100 na do 7 izolowanych instancji. Jest to decyzja architektoniczna i administracyjna dotycząca polityki przydziału zasobów, a jej konfiguracja wymaga wyłączenia węzła. Nie jest zadaniem programistycznym.

Wtyczki SPANK w C i skrypty job_submit w LUA to zaawansowane mechanizmy Slurma do modyfikowania środowiska zadań. Błąd w skrypcie job_submit blokuje wszystkie zgłoszenia zadań na klastrze. Ryzyko jest nieakceptowalne bez dedykowanego środowiska testowego odizolowanego od produkcji.

InfluxDB jako osobna baza metryk — odrzucona na podstawie analizy opisanej w sekcji 4. PostgreSQL + TimescaleDB pokrywa wszystkie przypadki użycia w jednym serwisie.

Modele predykcyjne zużycia energii wymagają co najmniej kilku miesięcy danych historycznych zanim cokolwiek ma sens statystyczny. To potencjalny projekt na kolejny semestr, po zebraniu danych przez wdrożony system.

--

8. Podsumowanie

Proponowana architektura zastępuje obecne rozwiązanie oparte na skryptach cron i plikach JSON systemem składającym się z czterech warstw: zbierania danych przez DCGM, przechowywania przez Prometheus (dane bieżące) i PostgreSQL+TimescaleDB (dane historyczne), analizy przez serwisy działające w tle i prezentacji przez Grafanę (administratorzy).

Kluczowe decyzje architektoniczne:

- DCGM zamiast nvidia-smi — eliminuje narzut i dostarcza natychmiastową korelację GPU↔zadanie
- **slurmdb** jako źródło prawdy o zadaniach — ufamy Slurm accounting zamiast reimplementować
- PostgreSQL + TimescaleDB jako jedyna baza danych — pokrywa zarówno relacyjne metadane jak i time-series bez konieczności utrzymywania dwóch systemów
- Retencja i downsampling zadeklarowane w SQL od pierwszego dnia — eliminuje problem niekontrolowanego wzrostu danych
- Estymator zasobów jako główna wartość dla użytkownika — przekształca historię zadań w konkretną sugestię **#SBATCH**